

# SETS

## Introduction to Sets

A set is an unordered, mutable collection used to store unique values.

Sets are created using `set()` function.

Sets automatically remove duplicate values.

Python

```
numbers = {10, 20, 30}
names = {"Ravi", "Teja", "Ankit"}
mixed = {10, "Python", 5.5, True}

print(numbers)
```

## SET PROPERTIES

| Property                          | Description                                             |
|-----------------------------------|---------------------------------------------------------|
| Unordered                         | Elements do not maintain insertion order                |
| Mutable                           | Elements can be added and removed                       |
| Unindexed                         | Elements cannot be accessed using index positions       |
| Iterable                          | Can iterate through elements using loops                |
| Unique Elements                   | Duplicate values are automatically removed              |
| Heterogeneous                     | Can store different datatypes                           |
| Dynamic Size                      | Elements can be added or removed anytime                |
| Supports Nested Immutable Objects | Can contain tuples and frozensets                       |
| Mutable Objects Not Allowed       | Lists, sets, and dictionaries cannot be stored directly |

# CREATING SETS

## Empty Set

```
Python  
s = set()
```

## Set with Duplicate Values

```
Python  
data = {10, 20, 30, 20, 10}  
  
print(data)          # {10, 20, 30}
```

Duplicates are removed automatically.

# SET OPERATIONS

## 1. Membership Operators

Checks whether an element exists.

```
Python  
data = {10, 20, 30}  
  
print(20 in data)      # True  
print(100 not in data) # True
```

## 2. Union

Combines all unique elements from both sets.

```
Python  
a = {1, 2, 3}  
b = {3, 4, 5}  
  
print(a | b)          # {1, 2, 3, 4, 5}
```

### 3. Intersection

Returns common elements.

Python

```
a = {1, 2, 3}
b = {2, 3, 4}

print(a & b)      # {2, 3}
```

### 4. Difference

Returns elements present in the first set but not second.

Python

```
a = {1, 2, 3}
b = {2, 3, 4}

print(a - b)      # {1}
```

### 5. Symmetric Difference

Returns elements present in either set but not both.

Python

```
a = {1, 2, 3}
b = {2, 3, 4}

print(a ^ b)      # {1, 4}
```

### 6. Subset

Checks whether all elements of one set exist in another.

Python

```
a = {1, 2}
b = {1, 2, 3, 4}

print(a <= b)      # True
```

## 7. Superset

Checks whether a set contains all elements of another set.

Python

```
a = {1, 2, 3, 4}
```

```
b = {1, 2}
```

```
print(a >= b)      # True
```

## BUILT-IN SET FUNCTIONS

| Function | Description                           | Example                   |
|----------|---------------------------------------|---------------------------|
| len()    | Returns number of elements            | len({1,2,3}) → 3          |
| max()    | Returns largest element               | max({10,20,5}) → 20       |
| min()    | Returns smallest element              | min({10,20,5}) → 5        |
| sum()    | Returns sum of numeric elements       | sum({1,2,3}) → 6          |
| sorted() | Returns sorted list                   | sorted({3,1,2}) → [1,2,3] |
| set()    | Converts iterable to set              | set("hello")              |
| any()    | Returns True if any element is True   | any({0,1})                |
| all()    | Returns True if all elements are True | all({1,2,3})              |

## SET METHODS

### 1. ADDING METHODS

| Method   | Description            | Example           |
|----------|------------------------|-------------------|
| add()    | Adds one element       | s.add(10)         |
| update() | Adds multiple elements | s.update([20,30]) |

## 2. REMOVING METHODS

| Method    | Description                         | Example       |
|-----------|-------------------------------------|---------------|
| remove()  | Removes element, error if absent    | s.remove(10)  |
| discard() | Removes element, no error if absent | s.discard(10) |
| pop()     | Removes a random element            | s.pop()       |
| clear()   | Removes all elements                | s.clear()     |

## 3. COPYING METHOD

| Method | Description          | Example  |
|--------|----------------------|----------|
| copy() | Creates shallow copy | s.copy() |

## 4. SET RELATION METHODS

| Method                 | Description            | Example                   |
|------------------------|------------------------|---------------------------|
| union()                | Combines sets          | a.union(b)                |
| intersection()         | Common elements        | a.intersection(b)         |
| difference()           | Elements in first only | a.difference(b)           |
| symmetric_difference() | Non-common elements    | a.symmetric_difference(b) |

## 5. SUBSET & SUPERSSET METHODS

| Method       | Description               | Example         |
|--------------|---------------------------|-----------------|
| issubset()   | Checks subset             | a.issubset(b)   |
| issuperset() | Checks superset           | a.issuperset(b) |
| isdisjoint() | Checks no common elements | a.isdisjoint(b) |

# FROZENSET

A frozenset is an immutable version of a set.

Once created, elements cannot be added or removed.

Python

```
data = frozenset({10, 20, 30})

print(data)
```

Properties of frozenset:

- Immutable
- Unordered
- Unique Elements
- Hashable
- Can be used inside sets and as dictionary keys

## WHY USE SETS?

- Remove duplicate values
- Perform mathematical set operations
- Fast membership checking
- Store unique items
- Compare collections efficiently

Examples:

Python

```
unique_ids = {101, 102, 103}

available_sizes = {"S", "M", "L", "XL"}

visited_pages = {"Home", "Products", "Cart"}
```

# DICTIONARIES

## Introduction to Dictionaries

A dictionary is a collection of key-value pairs used to store related data.

Dictionaries are created using curly braces { } or the dict() function.

Python

```
student = {  
    "id": 101,  
    "name": "Ravi",  
    "course": "Python"  
}
```

## DICTIONARY PROPERTIES

| Property                    | Description                                         |
|-----------------------------|-----------------------------------------------------|
| Ordered                     | Maintains insertion order (Python 3.7+)             |
| Mutable                     | Items can be added, removed, and modified           |
| Indexed by Keys             | Values are accessed using keys, not numeric indexes |
| Iterable                    | Can iterate through keys, values, and items         |
| Unique Keys                 | Duplicate keys are not allowed                      |
| Duplicate Values Allowed    | Multiple keys can have the same value               |
| Heterogeneous               | Can store different datatypes                       |
| Dynamic Size                | Can grow or shrink during execution                 |
| Nested Structures Supported | Can contain lists, tuples, sets, dictionaries, etc. |
| Keys Must Be Hashable       | Keys must be immutable/hashable objects             |

# CREATING DICTIONARIES

## Empty Dictionary

```
Python
data = {}
data = dict()
```

## Dictionary with Values

```
Python
student = {
    "id": 101,
    "name": "Ravi",
    "course": "Python"
}
```

## Using dict()

```
Python
student = dict(
    id=101,
    name="Ravi",
    course="Python"
)
```

# DICTIONARY OPERATIONS

## 1. Accessing Values

Values are accessed using keys.

```
Python
student = {
    "name": "Ravi",
    "age": 22
}
print(student["name"])    # Ravi
print(student["age"])    # 22
```

## 2. Updating Values

```
Python
student = {
    "name": "Ravi",
    "age": 22
}
student["age"] = 23
print(student)
```

## 3. Adding New Key-Value Pairs

```
Python
student = { "name": "Ravi"}
student["course"] = "Python"
print(student)
```

## 4. Removing Items

```
Python
student = {
    "name": "Ravi",
    "age": 22
}
del student["age"]
print(student)
```

## 5. Membership Operators

Membership checks only keys.

```
Python
student = {
    "name": "Ravi",
    "age": 22
}
print("name" in student)          # True
print("course" not in student)    # True
```

## BUILT-IN DICTIONARY FUNCTIONS

| Function | Description                       | Example   |
|----------|-----------------------------------|-----------|
| len()    | Returns number of key-value pairs | len(d)    |
| max()    | Returns maximum key               | max(d)    |
| min()    | Returns minimum key               | min(d)    |
| sorted() | Returns sorted list of keys       | sorted(d) |
| dict()   | Creates dictionary                | dict()    |
| any()    | Returns True if any key is True   | any(d)    |
| all()    | Returns True if all keys are True | all(d)    |

## DICTIONARY METHODS

### 1. ACCESS METHODS

| Method   | Description             | Example       |
|----------|-------------------------|---------------|
| get()    | Returns value for key   | d.get("name") |
| keys()   | Returns all keys        | d.keys()      |
| values() | Returns all values      | d.values()    |
| items()  | Returns key-value pairs | d.items()     |

### 2. ADDING & UPDATING METHODS

| Method       | Description                          | Example                    |
|--------------|--------------------------------------|----------------------------|
| update()     | Updates dictionary                   | d.update({"age":25})       |
| setdefault() | Returns value; creates key if absent | d.setdefault("city","Hyd") |

### 3. REMOVING METHODS

| Method | Description           | Example      |
|--------|-----------------------|--------------|
| pop()  | Removes specified key | d.pop("age") |

|           |                            |             |
|-----------|----------------------------|-------------|
| popitem() | Removes last inserted item | d.popitem() |
| clear()   | Removes all items          | d.clear()   |

## 4. COPYING METHODS

| Method | Description          | Example  |
|--------|----------------------|----------|
| copy() | Creates shallow copy | d.copy() |

## 5. CREATION METHODS

| Method     | Description                  | Example                    |
|------------|------------------------------|----------------------------|
| fromkeys() | Creates dictionary from keys | dict.fromkeys(["a","b"],0) |

## NESTED DICTIONARIES

A dictionary inside another dictionary is called a nested dictionary.

Python

```
students = {
    "s1": {"name": "Ravi", "age": 22 },
    "s2": {"name": "Teja", "age": 21}
}
print(students["s1"]["name"])    # Ravi
```

## MUTABLE VALUES INSIDE DICTIONARIES

Dictionary values can be mutable objects.

Python

```
student = {
    "marks": [90, 85, 88]
}
student["marks"].append(95)
print(student)

{'marks': [90, 85, 88, 95]}
```

# VALID DICTIONARY KEYS

## Allowed Keys

Python

```
data = {  
    101: "Ravi",  
    3.14: "Pi",  
    True: "Yes",  
    "name": "Python",  
    (1,2): "Tuple Key"  
}
```

## WHY USE DICTIONARIES?

- Store related information together
- Fast data retrieval using keys
- Represent real-world objects
- Flexible data structures
- Commonly used in APIs, JSON, databases, and applications

Examples:

Python

```
student = {  
    "id": 101,  
    "name": "Ravi",  
    "course": "Python"  
}  
  
product = {  
    "name": "Laptop",  
    "price": 50000  
}
```